Remote sensing in rangeland fire ecology: Comparing imagery to measured fire behavior, and burn severity across prescribed burns and wildfires

Supplementary Information: script for R and Copernicus browser

DA McGranahan

Contents

Setup **1**
- Study region . . . 1
- Data aquisition . . . 2
- Regional precipitation and anomalies . . . 2
- Wildfire perimeters . . . 4

Remotely-sensed data **6**
- Retrieving imagery . . . 6
- PIF corrections and dNBR calculations . . . 6
- Extracting severity data from rasters . . . 18

Statistical analysis **21**
- Space-based vs. field-based data . . . 21
- Wildfires vs. Rx burns . . . 21

This document is organized around the principal steps used to wrangle and analyze data in this project, and summarizes the R script used to perform each step. Script is provided for readers to understand the means by which data were sourced, assembled, and analyzed to better contextualize the results and their interpretations in the main manuscript. While in some cases it might serve as a resource for solutions to the data wrangling and analysis problems encountered in this project, is not presented as a linear, start-to-finish set of reproducible code. Some steps rely on data manipulated externally in QGIS.

Setup

```
# Load necessary packages via pacman utility
  pacman::p_load(tidyverse, magrittr, readxl, # data wrangling
                 foreach, doSNOW,             # parallel processing
                 sf, terra,                   # Spatial data
                 tigris,                      # Access US census data
                 climateR,                    # Fetch TerraClim data
                 lme4, emmeans       )        # statistical analysis
```

Study region

```r
# bounding box for focal area of study
  region_box <-
    tibble(feature = 'region',
           Easting = c(-900000, -215000, -215000, -900000),
           Northing = c(2470000, 2950000, 2470000, 2950000) ) %>%
    st_as_sf(coords = c("Easting", "Northing"), crs = 102039) %>%
    group_by(feature) %>%
    summarize(geometry = st_combine(geometry)) %>%
    st_cast("POLYGON") %>%
    st_bbox() %>%
    st_as_sfc(st_bbox(.))
# EPA Level III ecoregions
# (https://www.epa.gov/eco-research/level-iii-and-iv-ecoregions-continental-united-states)
  ngp <-
    read_sf(local_dir, # downloaded to local directory
            "us_eco_l3_state_boundaries") %>%
      filter(NA_L3NAME %in% c('Northwestern Glaciated Plains',
                              'Northwestern Great Plains')) %>%
      select(NA_L3NAME, STATE_NAME) %>%
      rename(L3 = NA_L3NAME, state = STATE_NAME) %>%
     st_crop(region_box)
# Get counties for the region
  ngp_counties <-
    tigris::counties(state = c('MT', "ND", 'SD'), cb = T) %>%
    st_transform(st_crs(ngp)) %>%
    st_intersection(ngp)
```

Data aquisition

Regional precipitation and anomalies

```r
# Create grid for region
  ngp_grid <-
    ngp %>%
      st_make_grid(cellsize = c(10000, 10000)) %>%
      st_intersection(ngp)  %>%
      st_as_sf() %>%
      rowid_to_column('cell')
# PReciptation data for study period
{
  # set up for parallel processing
  cores= parallel::detectCores()
  cl <- makeCluster(cores)
  registerDoSNOW(cl )

  begin = Sys.time()
  ngp_ppt <-
    foreach(i=1:length(ngp_grid$cell),
            .combine = bind_rows,
            .errorhandling = 'remove',
            .inorder = FALSE,
            .packages = c('tidyverse', 'sf', 'climateR')) %dopar% {
              getTerraClim(
```

```
                AOI = ngp_grid %>%
                      slice(i) %>%
                       st_centroid() ,
                varname = 'ppt',
                startDate = '2017-01-01',
                endDate = '2024-12-31'  ) %>%
                as_tibble() %>%
                separate(date, into = c('year', 'month', 'day'), sep = "-") %>%
                 group_by(year) %>%
                summarize(ppt = sum(ppt_total, na.rm = TRUE)) %>%
                summarize(ppt = mean(ppt, na.rm = TRUE)) %>%
                add_column(cell = i)
            }
  stopCluster(cl)
  Sys.time() - begin
}

  ppt_dat <- lst()
  ppt_grd <-
    ngp_grid  %>%
      merge(by = 'cell',
                ngp_ppt)
  ppt_dat$ppt_grd <- ppt_grd
# Find precip for counties with Research Extension Centers
  ppt_dat$REC_PPT <-
  ppt_grd %>%
    st_intersection(ngp_counties %>%
                      filter(STUSPS == 'ND',
                             NAME %in% c('Stutsman', 'Kidder', 'Adams')) %>%
                      select(L3) ) %>%
  filter(st_geometry_type(., by_geometry = TRUE) %in% c('MULTIPOLYGON', 'POLYGON'))  %>%
    as_tibble() %>%
    group_by(L3) %>%
    summarize(PPT_Mean = mean(ppt),
              PPT_SD = sd(ppt))
# Assign anomalies to grid
  SDs = 3 # define anomaly by standard deviation
  ppt_dat$anomaly <-
    ppt_grd %>%
      st_intersection(ngp, .) %>%
      filter(st_geometry_type(., by_geometry=TRUE) %in% c('MULTIPOLYGON','POLYGON')) %>%
      mutate(anomaly = case_when(
        L3 == "Northwestern Glaciated Plains" ~ ppt - 447,
        L3 == "Northwestern Great Plains" ~ ppt - 431 ),
        anom_cat = case_when(
          L3 == "Northwestern Glaciated Plains" &
            abs(anomaly) > 12 * SDs ~ 'beyond',
          L3 == "Northwestern Glaciated Plains" &
            abs(anomaly) < 12 * SDs ~ 'within',
          L3 == "Northwestern Great Plains" &
            abs(anomaly) > 19 * SDs ~ 'beyond',
          L3 == "Northwestern Great Plains" &
            abs(anomaly) < 19 * SDs ~ 'within') )
```

Wildfire perimeters

```
# Get fire perimeters for the study region by L3
  # https://data-nifc.opendata.arcgis.com/datasets/
  # nifc::interagencyfireperimeterhistory-all-years-view/
region_perims2 <-
  read_sf(local_dir, # downloaded to local directory
          "InterAgencyFirePerimeterHistory_All_Years_View") %>%
    select(FIRE_YEAR, INCIDENT, FEATURE_CA) %>%
    st_transform(st_crs(l3)) %>%
    st_make_valid(.) %>%
    st_intersection(l3) %>%
    rename(FireYear = FIRE_YEAR, FireName = INCIDENT, FireType = FEATURE_CA)

region_perims <-
  region_perims2  %>%
  mutate(FireType = case_when(
          str_sub(FireType, 1,4)=='Wild' ~ 'Wildfire',
          TRUE ~ 'RxFire' )) %>%
  filter(FireType == 'Wildfire',
         between(as.numeric(FireYear), 2017, 2024),
         state != "Wyoming") %>%
  mutate(FireName = case_when(
              FireName == 'nd-crr-fy22-wf-knutsen' ~ 'Knutsen',
              TRUE ~ FireName )) %>%
  mutate(FireCode = str_remove_all(FireName, ' Fire'),
         FireCode = gsub(pattern     = "[^a-z,A-Z,0-9]",
                         replacement = "",
                         FireCode),
         FireCode = paste0(FireCode, '_', FireYear),
         Ha = st_area(.),
         Ha = as.numeric(Ha) * 0.0001) %>%
  group_by(FireCode, FireYear, FireName, FireType, L3, state) %>%
  summarize(AreaHa = sum(Ha),
            .groups ='drop')  %>%
  group_by(FireCode) %>%
  arrange(desc(AreaHa)) %>%
  slice(1) %>% # in case fire crosses ecoregion/state lines
  ungroup()
# Identify wildfires within the precipitation anomaly
  win_fires <-
    region_perims %>%
      st_centroid() %>%
      st_intersection(ppt_dat$anomaly) %>%
      filter(anom_cat == 'within',
             AreaHa > 10) %>%
    as_tibble() %>%
    select(FireCode, L3, state, anomaly)

  precip_perims <-
    region_perims %>%
    filter(FireCode %in% win_fires$FireCode)
# Get rangeland classifications
  # Locally-saved USFS Extent of US Rangelands raster product
```

```
    # https://apps.fs.usda.gov/arcx/rest/services/
    # RDW_LandscapeAndWildlife/Extent_of_US_Rangelands/MapServer
  rr_tr <- terra::rast(paste0(local_dir, "/USrangelands.tif")) %>%
            terra::crop(st_transform(region_perims, 5070))
# Find the proportion rangeland for each fire
  range_fires <- tibble()
  for(i in 1:length(unique(precip_perims$FireCode))){
    fire = unique(precip_perims$FireCode)[i]
    precip_perims %>%
      filter(FireCode == fire )%>%
      st_transform(st_crs(rr_tr)) %>%
      terra::extract(rr_tr, .) %>%
      group_by(LABEL) %>%
      summarize(pixels = n() ) %>%
      mutate(prop = pixels / sum(pixels)) %>%
      add_column(FireCode = fire, .before = 1) %>%
      filter(LABEL == 'Rangeland') %>%
      bind_rows(range_fires) -> range_fires }
  range_fires %<>%
    mutate(RangeHa = (pixels * 900)* 0.0001) %>%
    select(-LABEL, -pixels)  %>%
    rename(PropRange = prop)
# create comparison wildfires object
  gp_perims <-
    bind_rows(
    # NW Glaciated Plains
      precip_perims %>%
        right_join(by = 'FireCode',
                   range_fires) %>%
        filter(L3 == 'Northwestern Glaciated Plains') %>%
        arrange(desc(PropRange)) %>%
        filter(PropRange > 0.2),
    # NW Great Plains
      precip_perims %>%
        right_join(by = 'FireCode',
                   range_fires) %>%
        merge(by ='FireCode',
              win_fires %>% select(FireCode, anomaly)) %>%
        filter(L3 == 'Northwestern Great Plains' &
               RangeHa > 10) %>%
        arrange((abs(anomaly)), desc(PropRange)) %>%
        slice(1:20) ) %>%
     mutate(type = 'Wildfire',
                zone = case_when(
                  FireCode %in% c('Swather_2021','Hwy31101STWest_2022', 'CB00121_2021',
                                  'CoalSeamWest_2021', '1806HunkpapaCreek_2021') ~
                    'east',
                  L3 == 'Northwestern Great Plains' ~ 'west',
                  L3 == 'Northwestern Glaciated Plains' ~ 'east'))
```

Remotely-sensed data

Retrieving imagery

This script was used to create and export rectangular Area of Interest (AOI) `.kml` files for each wildfire for input into Copernicus browser.

```
  for(i in 1:length(unique(gp_perims$FireCode))) {
    wd = "./AOIs"
    fire = unique(gp_perims$FireCode)[i]
    gp_perims %>%
      filter(FireCode == fire) %>%
      st_transform(4326) %>%
      st_bbox() %>%
      st_as_sfc() %>%
      st_write(., paste0(wd, '/', fire, '.kml'), append = FALSE )}
```

This EvalScript for the Copernicus browser exports raw band data required for Psuedo-Invariant Feature correction and Normalized Burn Ratio (NBR) calculation.

```
//VERSION=3
function setup() {
  return {
    input: ["B03", "B04", "B08", "B12"],
    output: { bands: 4, sampleType: "UINT16" }
  };
}

function evaluatePixel(sample) {
  let green = sample.B03;
  let red = sample.B04;
 let nir = sample.B08;
  let swir = sample.B12;
  // apply offset for UINT16
  return [10000 * green + 10000,
          10000 * red + 10000,
      10000 * nir + 10000,
          10000 * swir + 10000];
}
```

PIF corrections and dNBR calculations

```
pacman::p_load(tidyverse, magrittr, readxl, sf, terra, tidyterra)

# Function to calculate NBR
    calc_nbr <- function(nir, swir) {
                  return((nir - swir) / (nir + swir)) }
# Locally-saved Sentinel-2 imagery
  rx_dir = 'C:/.../Sentinel/Rx'
```

Prescribed burns

```
# Load Rx fire data
```

```
  # burn & imagery dates
    rx_dates <-read_xlsx("S:/DevanMcG/FireScience/Sentinel/SeverityComparison/ImageDates.xlsx",
                         'PreBurnPostDatesRxND') %>%
                mutate(across(c(BurnDate:PostBurn), ~ as.character(.x)))
# spatial & fire data for the thermocouples
  rx_sf <- read_sf('./paper/SpatialData.gpkg', 'RxBurnData')

# Imagery
  rx_dir = 'C:/.../SeverityComparison/Rx'

  rx_images <-
    tibble(file = list.files(rx_dir, pattern = "\\.tiff$", ignore.case = TRUE) ) %>%
    mutate(info = str_remove(file, '.tiff')) %>%
    separate(info, into = c('location', 'ImageDate', 'type'), sep = '_') %>%
    mutate(location = replace_values(location, 'DAY' ~ 'Dayton'))

  RxSeverity <- tibble()

##
## Location-specific corrected dNBR rasters
##

# Change manually and send loop on its way
  loc = 'CGREC'
  loc = 'HREC'
  loc = 'Dayton'

  begin = Sys.time()
  rx_d <- filter(rx_dates, location == loc)
  lc = tolower(loc)
  loc_sf <- st_read('./data/OriginalRxBurnPerims.gpkg', lc, quiet = TRUE) %>%
            st_transform(4326)
  tc_zones <- filter(rx_sf, location == loc) %>%
              mutate(area = st_area(.) ) %>%
              group_by(burn) %>%
              summarise(area = sum(area))  %>%
              st_centroid() %>%
              st_buffer(100)

  dNBR <- list()
    for(i in 1:length(rx_d$BurnDate)) {
      date = slice(rx_d, i)

    #
    # Step 1: Create cloud & shadow mask from Scene Classification Layers
    #
    # Load, wrangle SCL as output by Copernicus GUI
      # Pre-fire SCL
        pre_scl = filter(rx_images, location == loc,
                                    type == 'SCL',
                                    ImageDate == date$PreBurn)$file
        PreSCL <- paste0(rx_dir, '/', pre_scl) %>%
                    rast()
```

```
  names(PreSCL) <- c('r', 'g', 'b', 'dm')
  PreSCL %<>%
    mutate(sc = case_when(
      r == 0 & g == 0 & b  == 0 ~ 0,
      r == 255 & g == 0 & b== 0 ~ 1,
      r == 47 & g == 47 & b == 47 ~ 2,
      r == 100 & g == 50 & b == 0 ~ 3,
      r == 0 & g == 161 & b == 0 ~ 4,         # 160 -> 161
      r == 255 & g == 231 & b == 90 ~ 5,      # 230 -> 231
      r == 0 & g == 0 & b == 255 ~ 6,
      r == 129 & g == 129 & b == 129 ~ 7,    #  128 -> 129
      r == 193 & g == 193 & b == 193 ~ 8,    #  192 -> 193
      r == 255 & g == 255 & b == 255 ~ 9,
      r == 100 & g == 201 & b == 255 ~ 10,  # g 200 -> 201
      r == 255 & g == 150 & b == 255 ~ 11,
      TRUE ~ 99 ),
      sc = as.ordered(sc)) %>%
    select(sc)
# Post fire SCL
  post_scl = filter(rx_images, location == loc,
                    type == 'SCL',
                    ImageDate == date$PostBurn)$file
  PostSCL <- paste0(rx_dir, '/', post_scl) %>%
             rast()
  names(PostSCL) <- c('r', 'g', 'b', 'dm')
  PostSCL %<>%
    mutate(sc = case_when(
      r == 0 & g == 0 & b  == 0 ~ 0,
      r == 255 & g == 0 & b== 0 ~ 1,
      r == 47 & g == 47 & b == 47 ~ 2,
      r == 100 & g == 50 & b == 0 ~ 3,
      r == 0 & g == 161 & b == 0 ~ 4,         # g 160 -> 161
      r == 255 & g == 231 & b == 90 ~ 5,      # g 230 -> 231
      r == 0 & g == 0 & b == 255 ~ 6,
      r == 129 & g == 129 & b == 129 ~ 7,    #  128 -> 129
      r == 193 & g == 193 & b == 193 ~ 8,    #  192 -> 193
      r == 255 & g == 255 & b == 255 ~ 9,
      r == 100 & g == 201 & b == 255 ~ 10,  # g 200 -> 201
      r == 255 & g == 150 & b == 255 ~ 11,
      TRUE ~ 99 ),
      sc = as.ordered(sc)) %>%
    select(sc)
  # Define classifications to exclude
    ex_sc <- c(0, 2, 3, 7:11) %>% as.ordered()
  # Get the excluded classifications for each image
    pre_mask <-
      PreSCL %>%
      mutate(sc = ifelse(sc %in% ex_sc, 1, 0))
    post_mask <-
      PostSCL %>%
      mutate(sc = ifelse(sc %in% ex_sc, 1, 0))

  # Combine pre and post masks
```

```
      CombinedMask <- pre_mask + post_mask
    # Add back in TC sample areas in case they got excluded
      CombinedMask %<>%  mutate(sc = ifelse(sc >= 1, 1, 0))

      tc_pixels <- tc_zones %>%
                rasterize(CombinedMask, background = 0) %>%
                mutate(layer = ifelse(layer == 1, -1, 0))
      CombinedMask2 <- CombinedMask + tc_pixels
      CombinedMask2 %<>% mutate(sc = ifelse(sc <= 0, 0, 1),
                                sc = ifelse(sc >= 1, NA, 1))
#
# Step 2: PIF correction on post-fire image
#
  ## Load raw bands
    # pre image
      pre_raw = filter(rx_images, location == loc,
                       type == 'RAW',
                       ImageDate == date$PreBurn)$file
      pre_path = paste0(rx_dir, '/', pre_raw)
      pre_ras <- rast(pre_path) %>%
                    mask(., CombinedMask2)
      pre_ras <-  (pre_ras-10000)/10000
      names(pre_ras) <- c('green', 'red', 'nir', 'swir')
    # post-fire
      post_raw = filter(rx_images, location == loc,
                        type == 'RAW',
                        ImageDate == date$PostBurn)$file
      post_path = paste0(rx_dir, '/', post_raw)
      post_ras <- rast(post_path) %>%
                    mask(., CombinedMask2)
      post_ras = (post_ras-10000)/10000
      names(post_ras) <- c('green', 'red', 'nir', 'swir')
  # PIF correction
    # Calculate the absolute difference for PIF-sensitive bands
      diff_img <- abs(pre_ras[[c('green', 'red', 'swir')]] -
                        post_ras[[c('green', 'red', 'swir')]])
    # Sum the differences across bands to get a "change magnitude" map
      mag <- sum(diff_img)
    # Define a threshold for "No Change" (e.g., the bottom 1% of pixels)
      thresh <- quantile(values(mag, na.rm=TRUE), probs = 0.01)
    # Create PIF mask
      pif_mask = ifel(mag >= thresh, NA, mag)
    # Ensure there are enough relatively-invariant pixels to do regression
      pif_pix <- values(pif_mask$sum, na.rm=TRUE) %>% length()
      if(pif_pix < 100) {
        thresh <- quantile(values(mag, na.rm=TRUE), probs = 0.1)
        pif_mask = ifel(mag >= thresh, NA, mag)
        pif_pix <- values(pif_mask$sum, na.rm=TRUE) %>% length()
      }
    # Apply mask to images
      ref_pifs <- mask(pre_ras, pif_mask)   # Pre-burn reference image
      subj_pifs <- mask(post_ras, pif_mask) # subject image to correct
## A PIF correction loop combining Jay's script and`landsat::PIF` example
```

```
      # stash list
        corr_bands <- list()
      # loop through layers to get PIF pixels
        for(l in 1:nlyr(pre_ras)) {
          # Get PIF values for the current band
            vals <- tibble(
                      ref_val = values(ref_pifs[[l]], na.rm=TRUE),
                      subj_val = values(subj_pifs[[l]], na.rm=TRUE)
                      )
          # Fit the linear model
            post.corr <- lm(vals[[1]] ~ vals[[2]])$coefficients
            m = post.corr[[2]]
            b = post.corr[[1]]

          # Apply correction to the entire subject band
            corr_bands[[l]] <- (m * post_ras[[l]]) + b
        }

      # Merge corrected bands back into a single SpatRaster & crop to pastures
        corr_rast <- rast(corr_bands ) %>%
                        crop(loc_sf )
      # crop the pre-burn image to pastures as well
        pre_burn <- pre_ras %>% crop(loc_sf)
        # post_burn <- post_ras %>% crop(loc_sf)   # raw
    #
    # Step 3: Calculate dNBR from pre- and corrected post imagery
    #
      # create NBR rasters for pre and corrected post
        nbr_pre <- calc_nbr(pre_burn$nir, pre_burn$swir )
        nbr_post <- calc_nbr(corr_rast$nir, corr_rast$swir) # post-fire, corrected
        # nbr_post_raw <- calc_nbr(post_burn$nir, post_burn$swir) # post-fire, raw

      # Calculate dNBR
        dnbr <- (nbr_pre - nbr_post) %>% rename(dNBR = nir)
        names(dnbr) <- date$BurnDate
        dNBR[[i]] <- dnbr
  }
        rast(dNBR ) %>%
          writeRaster(paste0('./data/RxRasters/TC/', loc, '.tiff'), overwrite=TRUE)
```

Thermocouple sample points

```
# Load Rx fire data
  rx_perims <- read_sf('./paper/SpatialData.gpkg', 'RxBurnPerimeters')

# Identify unique post-burn dates to calculate severity for
  rx_perims %<>%
    select(location, burn, PostBurn, PreBurn) %>%
    mutate(location = case_when(
                        location %in% c('Fitch', 'Clement') ~ 'HREC',
                        TRUE ~ location
                      )) %>%
    arrange(location, PostBurn) %>%
```

```
    mutate(DeltaPair = paste0(PostBurn, '_', PreBurn))

  DeltaPairs <-
    rx_perims %>%
        as_tibble() %>%
      group_by(location) %>%
      distinct(DeltaPair) %>%
        ungroup() %>%
      separate(DeltaPair, into = c("PostBurn", "PreBurn"), sep = '_', remove = F )

  rx_images <-
    tibble(file = list.files(rx_dir, pattern = "\\.tiff$", ignore.case = TRUE) ) %>%
    mutate(info = str_remove(file, '.tiff')) %>%
    separate(info, into = c('location', 'ImageDate', 'type'), sep = '_') %>%
    mutate(location = replace_values(location, 'DAY' ~ 'Dayton'))
##
## Location-specific corrected dNBR rasters
##
  for(i in 1:length(unique(DeltaPairs$location))) {
    loc = unique(DeltaPairs$location)[i]
    dp_d <- filter(DeltaPairs, location == loc)
    loc_sf <- rx_perims %>% filter(location == loc) # for cropping dNBR raster
    dNBR <- list()
    cat(paste('Starting', loc, '...'))
    for(j in 1:length(dp_d$DeltaPair)) {
      dp = slice(dp_d, j)

    #
    # Step 1: Create cloud & shadow mask from Scene Classification Layers
    #
    # Load, wrangle SCL as output by Copernicus GUI
      # Pre-fire SCL
        pre_scl = filter(rx_images, location == dp$location,
                                    type == 'SCL',
                                    ImageDate == dp$PreBurn)$file
        PreSCL <- paste0(rx_dir, '/', pre_scl) %>%
                    rast()
        names(PreSCL) <- c('r', 'g', 'b', 'dm')
        PreSCL %<>%
          mutate(sc = case_when(
            r == 0 & g == 0 & b  == 0 ~ 0,
            r == 255 & g == 0 & b== 0 ~ 1,
            r == 47 & g == 47 & b == 47 ~ 2,
            r == 100 & g == 50 & b == 0 ~ 3,
            r == 0 & g == 161 & b == 0 ~ 4,         # 160 -> 161
            r == 255 & g == 231 & b == 90 ~ 5,      # 230 -> 231
            r == 0 & g == 0 & b == 255 ~ 6,
            r == 129 & g == 129 & b == 129 ~ 7,    #  128 -> 129
            r == 193 & g == 193 & b == 193 ~ 8,    #  192 -> 193
            r == 255 & g == 255 & b == 255 ~ 9,
            r == 100 & g == 201 & b == 255 ~ 10,  # g 200 -> 201
            r == 255 & g == 150 & b == 255 ~ 11,
            TRUE ~ 99 ),
```

```
        sc = as.ordered(sc)) %>%
      select(sc)
  # Post fire SCL
    post_scl = filter(rx_images, location == dp$location,
                      type == 'SCL',
                      ImageDate == dp$PostBurn)$file
    PostSCL <- paste0(rx_dir, '/', post_scl) %>%
               rast()
    names(PostSCL) <- c('r', 'g', 'b', 'dm')
    PostSCL %<>%
      mutate(sc = case_when(
        r == 0 & g == 0 & b  == 0 ~ 0,
        r == 255 & g == 0 & b== 0 ~ 1,
        r == 47 & g == 47 & b == 47 ~ 2,
        r == 100 & g == 50 & b == 0 ~ 3,
        r == 0 & g == 161 & b == 0 ~ 4,         # g 160 -> 161
        r == 255 & g == 231 & b == 90 ~ 5,      # g 230 -> 231
        r == 0 & g == 0 & b == 255 ~ 6,
        r == 129 & g == 129 & b == 129 ~ 7,    #  128 -> 129
        r == 193 & g == 193 & b == 193 ~ 8,    #  192 -> 193
        r == 255 & g == 255 & b == 255 ~ 9,
        r == 100 & g == 201 & b == 255 ~ 10,   # g 200 -> 201
        r == 255 & g == 150 & b == 255 ~ 11,
        TRUE ~ 99 ),
        sc = as.ordered(sc)) %>%
      select(sc)
    # Define classifications to exclude
      ex_sc <- c(0, 2, 3, 7:11) %>% as.ordered()
    # Get the excluded classifications for each image
      pre_mask <-
        PreSCL %>%
        mutate(sc = ifelse(sc %in% ex_sc, 1, 0))
      post_mask <-
        PostSCL %>%
        mutate(sc = ifelse(sc %in% ex_sc, 1, 0))
    # Combine pre and post masks
      CombinedMask <- pre_mask + post_mask
      CombinedMask %<>% mutate(sc = ifelse(sc >= 1, NA, 1))
#
# Step 2: PIF correction on post-fire image
#
  ## Load raw bands
    # pre image
      pre_raw = filter(rx_images, location == dp$location,
                       type == 'RAW',
                       ImageDate == dp$PreBurn)$file
      pre_path = paste0(rx_dir, '/', pre_raw)
      pre_ras <- rast(pre_path) %>%
                   mask(., CombinedMask)
      pre_ras <-  (pre_ras-10000)/10000
      names(pre_ras) <- c('green', 'red', 'nir', 'swir')
    # post-fire
      post_raw = filter(rx_images, location == dp$location,
```

```
                        type == 'RAW',
                        ImageDate == dp$PostBurn)$file
      post_path = paste0(rx_dir, '/', post_raw)
      post_ras <- rast(post_path) %>%
                    mask(., CombinedMask)
      post_ras = (post_ras-10000)/10000
      names(post_ras) <- c('green', 'red', 'nir', 'swir')
  # PIF correction
    # Calculate the absolute difference for PIF-sensitive bands
      diff_img <- abs(pre_ras[[c('green', 'red', 'swir')]] -
                        post_ras[[c('green', 'red', 'swir')]])
    # Sum the differences across bands to get a "change magnitude" map
      mag <- sum(diff_img)
    # Define a threshold for "No Change" (e.g., the bottom 1% of pixels)
      thresh <- quantile(values(mag, na.rm=TRUE), probs = 0.01)
    # Create PIF mask
      pif_mask = ifel(mag >= thresh, NA, mag)
    # Ensure there are enough relatively-invariant pixels to do regression
      pif_pix <- values(pif_mask$sum, na.rm=TRUE) %>% length()
      if(pif_pix < 100) {
        thresh <- quantile(values(mag, na.rm=TRUE), probs = 0.1)
        pif_mask = ifel(mag >= thresh, NA, mag)
        pif_pix <- values(pif_mask$sum, na.rm=TRUE) %>% length()
      }
    # Apply mask to images
      ref_pifs <- mask(pre_ras, pif_mask)   # Pre-burn reference image
      subj_pifs <- mask(post_ras, pif_mask) # subject image to correct
## PIF correction loop
  # stash list
    corr_bands <- list()
  # loop through layers to get PIF pixels
    for(l in 1:nlyr(pre_ras)) {
      # Get PIF values for the current band
        vals <- tibble(
                  ref_val = values(ref_pifs[[l]], na.rm=TRUE),
                  subj_val = values(subj_pifs[[l]], na.rm=TRUE)
                  )
      # Fit the linear model
        post.corr <- lm(vals[[1]] ~ vals[[2]])$coefficients
        m = post.corr[[2]]
        b = post.corr[[1]]

      # Apply correction to the entire subject band
        corr_bands[[l]] <- (m * post_ras[[l]]) + b
    } # close band correction loop

  # Merge corrected bands back into a single SpatRaster & crop to pastures
    corr_rast <- rast(corr_bands ) %>%
                    crop(loc_sf )
  # crop the pre-burn image to pastures as well
    pre_burn <- pre_ras %>% crop(loc_sf)
    # post_burn <- post_ras %>% crop(loc_sf)   # raw
#
```

```
    # Step 3: Calculate dNBR from pre- and corrected post imagery
    #
      # create NBR rasters for pre and corrected post
        nbr_pre <- calc_nbr(pre_burn$nir, pre_burn$swir )
        nbr_post <- calc_nbr(corr_rast$nir, corr_rast$swir) # post-fire, corrected
        # nbr_post_raw <- calc_nbr(post_burn$nir, post_burn$swir) # post-fire, raw

      # Calculate dNBR
        dnbr <- (nbr_pre - nbr_post) %>% rename(dNBR = nir)
        names(dnbr) <- dp$DeltaPair
        dNBR[[j]] <- dnbr
    } # close DeltaPair loop
    rast(dNBR ) %>%
      writeRaster(paste0('./data/RxRasters/Pasture/', loc, '.tiff'), overwrite=TRUE)
  } # close location loop
```

Entire pastures

Wildfires

```
# Regional rangeland raster from local version of USFS Rangelands product
# https://data.fs.usda.gov/geodata/rastergateway/rangelands/index.php
  rr <- terra::rast("C:/.../USrangelands.tif")
  region <- read_sf('./data/SeverityComparison.gpkg', 'StudyRegion') %>%
              st_transform(st_crs(rr))

  r_rr <- rr %>% terra::crop(region)

  wf_sf <- read_sf('./data/SeverityComparison.gpkg', 'GreatPlainsModifiedWF')

  wf_dir = '...'

  wf_images <-
    tibble(file = list.files(wf_dir, pattern = "\\.tiff$", ignore.case = TRUE) ) %>%
    mutate(info = str_remove(file, '.tiff')) %>%
    separate(info, into = c('FireCode', 'period', 'ImageDate', 'type'), sep = '_') %>%
    mutate(FireCode = str_replace(FireCode, '-', '_'))

  wf_perims <- wf_sf %>%
              filter(FireCode %in% unique(wf_images$FireCode))

  WfSeverity <- tibble()

    for(i in 1:length(unique(wf_perims$FireCode))) {
    # Get fire
      fc = unique(wf_perims$FireCode)[i]
      fire <-  wf_perims %>%
                filter(FireCode == fc)

    #
    # Step 1: Create cloud & shadow mask from Scene Classification Layers
    #
    # Load, wrangle SCL as output by Copernicus GUI
```

```
  # Pre-fire SCL
    pre_scl = filter(wf_images, FireCode == fc, type == 'SCL',
                     period == 'B')$file
    PreSCL <- paste0(wf_dir, '/', pre_scl) %>%
                  rast()
    names(PreSCL) <- c('r', 'g', 'b', 'dm')
    PreSCL %<>%
      mutate(sc = case_when(
        r == 0 & g == 0 & b  == 0 ~ 0,
        r == 255 & g == 0 & b== 0 ~ 1,
        r == 47 & g == 47 & b == 47 ~ 2,
        r == 100 & g == 50 & b == 0 ~ 3,
        r == 0 & g == 161 & b == 0 ~ 4,         # 160 -> 161
        r == 255 & g == 231 & b == 90 ~ 5,      # 230 -> 231
        r == 0 & g == 0 & b == 255 ~ 6,
        r == 129 & g == 129 & b == 129 ~ 7,    #  128 -> 129
        r == 193 & g == 193 & b == 193 ~ 8,    #  192 -> 193
        r == 255 & g == 255 & b == 255 ~ 9,
        r == 100 & g == 201 & b == 255 ~ 10,  # g 200 -> 201
        r == 255 & g == 150 & b == 255 ~ 11,
        TRUE ~ 99 ),
        sc = as.ordered(sc)) %>%
      select(sc)
  # Post fire SCL
    post_scl = filter(wf_images, FireCode == fc, type == 'SCL',
                     period == 'A')$file
    PostSCL <- paste0(wf_dir, '/', post_scl) %>%
                 rast()
    names(PostSCL) <- c('r', 'g', 'b', 'dm')
    PostSCL %<>%
      mutate(sc = case_when(
        r == 0 & g == 0 & b  == 0 ~ 0,
        r == 255 & g == 0 & b== 0 ~ 1,
        r == 47 & g == 47 & b == 47 ~ 2,
        r == 100 & g == 50 & b == 0 ~ 3,
        r == 0 & g == 161 & b == 0 ~ 4,         # g 160 -> 161
        r == 255 & g == 231 & b == 90 ~ 5,      # g 230 -> 231
        r == 0 & g == 0 & b == 255 ~ 6,
        r == 129 & g == 129 & b == 129 ~ 7,    #  128 -> 129
        r == 193 & g == 193 & b == 193 ~ 8,    #  192 -> 193
        r == 255 & g == 255 & b == 255 ~ 9,
        r == 100 & g == 201 & b == 255 ~ 10,  # g 200 -> 201
        r == 255 & g == 150 & b == 255 ~ 11,
        TRUE ~ 99 ),
        sc = as.ordered(sc)) %>%
      select(sc)
    # Define classifications to exclude
      ex_sc <- c(0, 2, 3, 7:11) %>% as.ordered()
    # Get the excluded classifications for each image
      pre_mask <-
        PreSCL %>%
        mutate(sc = ifelse(sc %in% ex_sc, 1, 0))
      post_mask <-
```

```
        PostSCL %>%
        mutate(sc = ifelse(sc %in% ex_sc, 1, 0))
    # Create the combined mask
      CombinedMask <- pre_mask + post_mask
      CombinedMask %<>% mutate(sc = ifelse(sc >= 1, NA, 1))
#
# Step 2: PIF correction on post-fire image
#
  ## Load raw bands
    # pre image
      pre_raw = filter(wf_images, FireCode == fc, type == 'RAW',
                       period == 'B')$file
      pre_path = paste0(wf_dir, '/', pre_raw)
      pre_ras <- rast(pre_path) %>%
                   mask(., CombinedMask)
      pre_ras <-  (pre_ras-10000)/10000
      names(pre_ras) <- c('green', 'red', 'nir', 'swir')
    # post-fire
      post_raw = filter(wf_images, FireCode == fc, type == 'RAW',
                        period == 'A')$file
      post_path = paste0(wf_dir, '/', post_raw)
      post_ras <- rast(post_path) %>%
                   mask(., CombinedMask)
      post_ras = (post_ras-10000)/10000
      names(post_ras) <- c('green', 'red', 'nir', 'swir')
  # PIF correction
    # Calculate the absolute difference for PIF-sensitive bands
      diff_img <- abs(pre_ras[[c('green', 'red', 'swir')]] -
                      post_ras[[c('green', 'red', 'swir')]])
    # Sum the differences across bands to get a "change magnitude" map
      mag <- sum(diff_img)
    # Define a threshold for "No Change" (e.g., the bottom 1% of pixels)
      thresh <- quantile(values(mag, na.rm=TRUE), probs = 0.01)
    # Create PIF mask
      pif_mask = ifel(mag >= thresh, NA, mag)
    # Ensure there are enough relatively-invariant pixels to do regression
      pif_pix <- values(pif_mask$sum, na.rm=TRUE) %>% length()
      if(pif_pix < 100) {
        thresh <- quantile(values(mag, na.rm=TRUE), probs = 0.1)
        pif_mask = ifel(mag >= thresh, NA, mag)
        pif_pix <- values(pif_mask$sum, na.rm=TRUE) %>% length()
      }
      if(pif_pix < 10) {
        thresh <- quantile(values(mag, na.rm=TRUE), probs = 0.1)
        pif_mask = ifel(mag >= thresh, NA, mag)
        pif_pix <- values(pif_mask$sum, na.rm=TRUE) %>% length()
      }

    # Apply mask to images
      ref_pifs <- mask(pre_ras, pif_mask)   # Pre-burn reference image
      subj_pifs <- mask(post_ras, pif_mask) # subject image to correct
## PIF correction loop
  # stash list
```

```
    corr_bands <- list()
  # loop through layers to get PIF pixels
    for(i in 1:nlyr(pre_ras)) {
      # Get PIF values for the current band
        vals <- tibble(
                  ref_val = values(ref_pifs[[i]], na.rm=TRUE),
                  subj_val = values(subj_pifs[[i]], na.rm=TRUE)
                  )
      # Fit the linear model
        # post.corr <- suppressMessages(
        #               lmodel2:::lmodel2(vals[[1]] ~ vals[[2]])$regression.results[2, 2:3]
        #               )
        post.corr <- lm(vals[[1]] ~ vals[[2]])$coefficients
        m = post.corr[[2]]
        b = post.corr[[1]]
      # Apply correction to the entire subject band
        corr_bands[[i]] <- (m * post_ras[[i]]) + b
      }

  # Merge corrected bands back into a single SpatRaster & crop to fire perimeter
    corr_rast <- rast(corr_bands ) %>%
                    crop(fire %>%
                           st_transform(4326)
                         )
  # crop the pre-burn image to the fire perimeter as well
    pre_burn <- pre_ras %>% crop(fire %>% st_transform(4326))
#
# Step 3: Calculate dNBR from pre- and corrected post imagery
#
  # create NBR rasters for pre and corrected post
    nbr_pre <- calc_nbr(pre_burn$nir, pre_burn$swir )
    nbr_post <- calc_nbr(corr_rast$nir, corr_rast$swir) # post-fire, corrected

  # Calculate dNBR
    dnbr <- (nbr_pre - nbr_post) %>% rename(dNBR = nir)
#
# Step 4: Sample dNBR from rangeland pixels
#
  # internal buffer against edge effects
    buff <- fire %>%
              st_transform(albersEAC) %>%
              mutate(AreaHa = st_area(.),
                     AreaHa = as.numeric(AreaHa) * 0.0001) %>%
              st_buffer(-20)
  # create gridded sample points
    # cell size factor scaled to total area
      cs = case_when(
            buff$AreaHa < 10 ~ 25,
            between(buff$AreaHa, 10, 50) ~ 36,
            between(buff$AreaHa, 50, 100) ~ 50,
            between(buff$AreaHa, 100, 200) ~ 100,
            between(buff$AreaHa, 200, 300) ~ 125,
            between(buff$AreaHa, 300, 500) ~ 150,
```

```
                between(buff$AreaHa, 500, 700) ~ 200,
                between(buff$AreaHa, 700, 3000) ~ 300,
                between(buff$AreaHa, 3000, 5000) ~ 500,
                between(buff$AreaHa, 5000, 7000) ~ 700,
                buff$AreaHa > 7000 ~ 1000 )
       # Find points as centroids of grid w/in perimeter
          pts <- buff %>%
                  st_make_grid(cellsize = cs,
                               square = FALSE) %>%
                  st_centroid() %>%
                  st_intersection(buff) %>%
                  st_as_sf() %>%
                  rowid_to_column('ID') %>%
                  st_transform(5070)
        # Filter sample points to rangeland cells
          r_pts <-
            terra::extract(r_rr,
                           terra::vect(pts),
                           df = TRUE) %>%
            filter(LABEL %in% c('Rangeland',
                                'Transitional Rangeland',
                                'Afforested CO') )
          pts %<>% filter(ID %in% r_pts$ID)
      # Extract values at points
        terra::extract(dnbr,
                       pts %>%
                         st_transform(4326) %>%
                         terra::vect(),
                       fun = mean,
                       method = 'bilinear',
                       bind = TRUE)  %>%
        as_tibble() %>%
        add_column(FireCode=fc, .before = 1 ) %>%
        filter(! is.na(dNBR)) %>%
        bind_rows(., WfSeverity) -> WfSeverity
    }
```

Extracting severity data from rasters

For thermocouple locations in Rx fires

```
# Load Rx fire data
  # burn & imagery dates
    rx_dates <-read_xlsx("C:/.../ImageDates.xlsx",
                         'PreBurnPostDatesTC') %>%
      mutate(across(c(BurnDate:PostBurn), ~ as.character(.x)))
  # spatial & fire data for the thermocouples
    rx_sf <- st_read('./data/ThermocouplePlacements.gpkg',
                     'AllBurnData') %>%
              mutate(BurnDate = as.character(date) ) %>%
              select(-date) %>%
              full_join(by = c('location', 'BurnDate'),
                        rx_dates) %>%
```

```
              st_transform(4326)

RxPointSeverity <- tibble()

  for(l in 1:length(unique(rx_perims$burn))) {
    lc = unique(rx_sf$location)[l]
    loc = filter(rx_sf, location == lc)
    dnbr_fp <- case_when( # locally-saved rasters created from above
                  lc == 'CGREC' ~ './data/RxRasters/TC/CGREC.tiff',
                  lc == 'HREC' ~ './data/RxRasters/TC/HREC.tiff',
                  lc == 'Dayton' ~ './data/RxRasters/TC/Dayton.tiff'
                )
    dnbr_rast <- rast(dnbr_fp)
  # loop for individual burns
    for(b in 1:length(unique(loc$burn))) {
      bn = unique(loc$burn)[b]
      #bn = 'DAY.1.19'
      burn = filter(loc, burn == bn)
      date = burn$BurnDate
      dnbr <- dnbr_rast %>% select(all_of(date))
      names(dnbr) <- 'dNBR'
          # Extract values at points
            terra::extract(dnbr,
                           burn ,
                           fun = mean,
                           method = 'bilinear',
                           bind = TRUE)  %>%
            as_tibble() %>%
              select(location:plot, BurnDate, MaxC:ros, dNBR) %>%
            bind_rows(., RxPointSeverity) -> RxPointSeverity
}
}
RxPointSeverity %<>%
  mutate(zone = ifelse(location == 'CGREC', 'East', 'West'))
```

Pasture-level Rx fires

```
# Load Rx fire data
  rx_perims <- read_sf('./data/SeverityComparison.gpkg', 'rx_perims')

  RxPastureSeverity <- tibble()

  for(b in 1:length(unique(rx_perims$burn))) {
    bn = unique(rx_perims$burn)[b]
    burn = filter(rx_perims, burn == bn)
    lc = burn$location
    # Map to and load severity raster for the location
      dnbr_fp <- case_when( # locally-saved rasters created from above
                    lc == 'CGREC' ~ './data/RxRasters/Pasture/CGREC.tiff',
                    lc == 'HREC' ~ './data/RxRasters/Pasture/HREC.tiff',
                    lc == 'Dayton' ~ './data/RxRasters/Pasture/Dayton.tiff'
                  )
      dnbr_rast <- rast(dnbr_fp)
```

```
    #
    # process the burn
    #
      dp = burn$DeltaPair
      dnbr <- dnbr_rast %>%
                select(all_of(dp)) %>%
                crop(burn)
      names(dnbr) <- 'dNBR'
    # create sample points
      buff <- burn %>%
                st_transform(albersEAC) %>%
                st_buffer(-20)
      pts <-  buff %>%
                st_make_grid(cellsize = c(diff(st_bbox(.)[c(1, 3)]),
                                          diff(st_bbox(.)[c(2, 4)]))/ 10) %>%
                st_centroid() %>%
                st_intersection(buff) %>%
                st_transform(4326)
    # Extract dNBR values at points
      terra::extract(dnbr,
                     pts %>%
                       vect() ,
                     fun = mean,
                     method = 'bilinear',
                     bind = TRUE)  %>%
      as_tibble() %>%
      tibble(location = lc,
             burn = bn,
             . ) %>%
      bind_rows(., RxPastureSeverity) -> RxPastureSeverity
}
```

Combine wildfire and Rx burn severity data

Wildfire severity extracted in same script that calculated it, above.

```
# Load data
   load('./data/RxPastureSeverity.Rdata')
   load('./data/WfSeverityLM.Rdata') # wildfire burn indices
   wf_sf <- read_sf('./data/SeverityComparison.gpkg',
                    'GreatPlainsModifiedWF') # Perimeter data

BurnSeverityData <-
  bind_rows(
    # Rx fires
    RxPastureSeverity %>%
      rename(FireCode = burn) %>%
        filter(complete.cases(.) ) %>%
        group_by(zone, location, FireCode) %>%
        summarise_at(.vars = vars(dNBR),
                     .funs = c(Mean_dNBR="mean",
                               SEM = function(x) {sd(x)/sqrt(n()) } )) %>%
        ungroup() %>%
        mutate(type = 'Rx') %>%
```

```
        select(type, zone, FireCode:SEM) ,
    # Wildfires
      wf_sf %>%
        as_tibble() %>%
        select(FireCode, L3) %>%
      right_join(by = 'FireCode',
        WfSeverityLM %>%
          group_by(FireCode) %>%
          summarise_at(.vars = vars(dNBR),
                       .funs = c(Mean_dNBR="mean",
                                 SEM = function(x) {sd(x)/sqrt(n()) } )) ) %>%
         mutate(type = 'Wildfire',
                zone = case_when(
                  FireCode %in% c('Swather_2021','Hwy31101STWest_2022', 'CB00121_2021',
                                  'CoalSeamWest_2021', '1806HunkpapaCreek_2021') ~
                    'East',
                  L3 == 'Northwestern Great Plains' ~ 'West',
                  L3 == 'Northwestern Glaciated Plains' ~ 'West')) %>%
      select(-L3) )
```

Statistical analysis

Space-based vs. field-based data

```
sev_dat <- readxl::read_xlsx('SeverityComparisonData.xlsx',
                             'PlotBurnIndices')
# Canopy temperature
  mc <- lme4::lmer(dNBR ~ MaxC + zone + (1|location/burn), REML = FALSE, data = sev_dat )

    car::Anova(mc)
    MuMIn::r.squaredGLMM(mc)

# Soil surface temperature
  sc <- lme4::lmer(dNBR ~ SoilMaxC + zone + (1|location/burn), REML = FALSE,
                   data = filter(sev_dat, !is.na(SoilMaxC)) )

  car::Anova(sc)
  MuMIn::r.squaredGLMM(sc)

# Rate of spread
  rs <- lme4::lmer(dNBR ~ ros + zone + (1|location/burn), REML = FALSE,
                   data = filter(sev_dat, ! is.na(ros)) )

  car::Anova(rs)
  MuMIn::r.squaredGLMM(rs)
```

Wildfires vs. Rx burns

```
reg_dat <- readxl::read_xlsx('SeverityComparisonData.xlsx',
                                  'BurnSeverityData') %>%
            rename(dNBR = dNBR_Mean) %>%
            select(zone, type, dNBR)
```

```r
sev_0 <- lm(dNBR ~ 1, data = reg_dat)
sev_int <- lm(dNBR ~ type*zone, data = reg_dat)

anova(sev_0, sev_int)

summary(sev_int)$r.squared

emmeans::joint_tests(type_comp, by = 'zone')
```